

# Relazione Finale

Progetto di Sistemi Operativi 2025/26  
Simulazione di una Mensa Universitaria

Versione 24 CFU

Ludovico Audenino

6 giugno 2026

## Indice

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                         | <b>3</b>  |
| <b>2</b> | <b>Architettura Multi-Processo</b>                          | <b>3</b>  |
| 2.1      | Ciclo di Vita del Coordinatore (Responsabile Mensa)         | 3         |
| 2.2      | Ciclo di Vita dell'Operatore                                | 5         |
| 2.3      | Ciclo di Vita dell'Utente                                   | 6         |
| 2.4      | Algoritmi di Scelta e Comportamento dell'Utente             | 6         |
| 2.4.1    | Scelta del Menù e Preferenze (Algoritmo Fisher-Yates)       | 6         |
| 2.4.2    | Accodamento Dinamico Bilanciato (Shortest Queue First)      | 7         |
| 2.4.3    | Comportamento in caso di Esaurimento Porzioni               | 7         |
| 2.4.4    | Comportamento in caso di Mancato Servizio (Not Served Path) | 8         |
| 2.4.5    | Consumazione al Tavolo                                      | 8         |
| 2.4.6    | Chiusura Anticipata della Giornata (Fine Servizio)          | 8         |
| <b>3</b> | <b>Meccanismi IPC Scelti e Motivazioni</b>                  | <b>9</b>  |
| 3.1      | Shared Memory                                               | 9         |
| 3.2      | Semafori                                                    | 9         |
| 3.3      | Code di Messaggi                                            | 10        |
| <b>4</b> | <b>Politica di Assegnazione Operatori</b>                   | <b>10</b> |
| 4.1      | Fase 1: Assegnazione Garantita                              | 11        |
| 4.2      | Fase 2: Distribuzione Bilanciata                            | 11        |
| 4.3      | Ordinamento per Priorità                                    | 11        |
| <b>5</b> | <b>Gestione della Sincronizzazione e della Concorrenza</b>  | <b>11</b> |
| 5.1      | Barriera di Sincronizzazione                                | 11        |
| 5.2      | Sezioni Critiche                                            | 12        |
| 5.3      | Gestione delle Risorse con Semafori Contatore               | 12        |
| 5.4      | Bilanciamento del Carico Utente                             | 12        |
| 5.5      | Pause Operatore                                             | 12        |

|           |                                              |           |
|-----------|----------------------------------------------|-----------|
| <b>6</b>  | <b>Gestione della Terminazione</b>           | <b>12</b> |
| 6.1       | Timeout (Causa 0) . . . . .                  | 12        |
| 6.2       | Overload (Causa 1) . . . . .                 | 13        |
| 6.3       | Segnale Esterno — SIGINT (Causa 2) . . . . . | 13        |
| 6.4       | Shutdown Graceful . . . . .                  | 13        |
| <b>7</b>  | <b>Struttura dei File di Configurazione</b>  | <b>13</b> |
| 7.1       | Formato Generale . . . . .                   | 13        |
| 7.2       | Parametri di Configurazione . . . . .        | 14        |
| 7.3       | File Menù . . . . .                          | 14        |
| 7.4       | Configurazioni di Consegna . . . . .         | 14        |
| <b>8</b>  | <b>Compilazione e Utilizzo</b>               | <b>14</b> |
| 8.1       | Requisiti . . . . .                          | 14        |
| 8.2       | Compilazione . . . . .                       | 15        |
| 8.3       | Esecuzione . . . . .                         | 15        |
| 8.4       | Pulizia Risorse IPC . . . . .                | 15        |
| <b>9</b>  | <b>Struttura del Progetto</b>                | <b>15</b> |
| <b>10</b> | <b>Conclusioni</b>                           | <b>16</b> |

# 1 Introduzione

Il progetto realizza una simulazione multi-processo di una mensa universitaria. Il sistema modella il flusso degli utenti attraverso diverse stazioni di servizio (primi piatti, secondi piatti, caffè/dolci, cassa) e la gestione degli operatori che servono i clienti. Tutti i processi comunicano esclusivamente tramite meccanismi IPC System V: **shared memory**, **semafori** e **code di messaggi**.

La simulazione è governata da un processo coordinatore (**responsabile\_mensa**) che orchestra il ciclo giornaliero, gestisce il rifornimento delle porzioni e monitora le condizioni di terminazione.

## 2 Architettura Multi-Processo

Il sistema è composto da tre tipologie di processi indipendenti, ciascuno compilato come eseguibile separato:

1. **responsabile\_mensa** — Il processo coordinatore (padre). Crea tutte le risorse IPC, legge la configurazione, genera i processi figli tramite **fork()** + **execve()**, e orchestra il ciclo di simulazione giorno per giorno.
2. **operatore** — Ogni istanza rappresenta un operatore assegnato a una specifica stazione. Riceve ordini dalla coda messaggi della propria stazione, simula il tempo di servizio, e invia la risposta all'utente.
3. **utente** — Ogni istanza rappresenta un cliente della mensa. Sceglie il menù, si mette in coda alle stazioni (bilanciando il carico), paga alla cassa e mangia al tavolo.

### 2.1 Ciclo di Vita del Coordinatore (Responsabile Mensa)

Il processo coordinatore (**responsabile\_mensa**) ha il compito di orchestrare l'intera simulazione, gestire le risorse condivise e verificare le condizioni di terminazione. Il suo ciclo di vita si articola nelle seguenti fasi:

1. **Inizializzazione delle Risorse IPC**: Crea il segmento di shared memory, alloca il set di semafori e inizializza le code di messaggi per ciascuna stazione. Carica i parametri dal file di configurazione e dal file del menù.
2. **Generazione dei Processi Figli**: Esegue la creazione di **NOF\_WORKERS** operatori e **NOF\_USERS** utenti. Per ciascun processo, effettua una **fork()** seguita da **execve()** per caricare gli eseguibili dedicati (**bin/operatore** e **bin/utente**), passando come argomenti l'identificatore della memoria condivisa ed eventuali parametri specifici (come la stazione target per l'operatore).
3. **Barriera di Avvio**: Attende che tutti i processi figli abbiano completato l'inizializzazione e segnalato la propria prontezza tramite l'incremento del semaforo di barriera **SEM\_READY**. Il coordinatore esegue un'operazione semaforica bloccante con valore pari a **-TOTAL\_CHILDREN**.
4. **Ciclo Giornaliero**: Per ciascuno dei **SIM\_DURATION** giorni di simulazione:

- a) **Apertura del Giorno:** Ripristina le code dei messaggi per la nuova giornata (per evitare messaggi pendenti del giorno prima) e reimposta a zero i contatori delle code.
  - b) **Sblocco dei Figli:** Esegue un incremento di massa pari a `+TOTAL_CHILDREN` sul semaforo `SEM_DAY_START`, consentendo a tutti i figli di iniziare le proprie routine giornaliere contemporaneamente.
  - c) **Rifornimento Periodico:** Durante la giornata, a intervalli regolari (es. ogni `AVG_REFILL_RATE` minuti simulati), simula l'arrivo di nuovi rifornimenti di cibo, incrementando le porzioni disponibili per i primi e secondi piatti in shared memory fino al limite massimo configurato (`MAX_PORZIONI_*`).
  - d) **Fine della Giornata:** Allo scadere del tempo simulato per la giornata, imposta il flag `day_running = 0` e invia il segnale `SIGUSR1` a tutti i figli per sbloccare le chiamate di ricezione messaggi bloccanti. Successivamente, attende che tutti i figli tornino in stato di pronto incrementando `SEM_READY`.
  - e) **Controllo Overload:** Verifica se il numero complessivo di utenti rimasti in coda supera la soglia di overload configurata (`OVERLOAD_THRESHOLD`). In caso affermativo, imposta la causa di terminazione ad Overload ed arresta la simulazione.
  - f) **Report Giornaliero:** Elabora le statistiche raccolte in shared memory relative alla giornata appena conclusa e le stampa a schermo.
5. **Shutdown e Pulizia:** Al termine del ciclo di simulazione (o in seguito a interruzioni da segnali esterni), imposta `simulation_running = 0`, rilascia i figli in attesa ed esegue una `waitpid()` su tutti i processi figli per evitarne la permanenza in stato zombie. Infine, distrugge tutte le risorse IPC create (shared memory, semafori, code di messaggi).

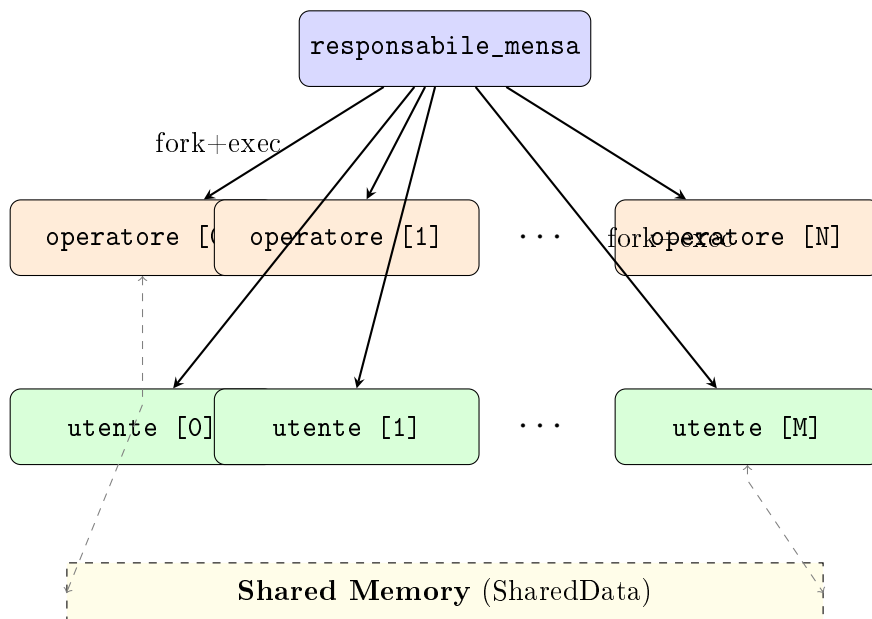


Figura 1: Architettura multi-processo della simulazione.

## 2.2 Ciclo di Vita dell'Operatore

I processi `operatore` rappresentano il personale addetto al servizio presso le stazioni di cibo (primi piatti, secondi piatti, caffè/dolci) o alla cassa per i pagamenti. Ciascun operatore esegue le seguenti fasi durante il suo ciclo di vita:

1. **Inizializzazione:** Riceve all'avvio (tramite argomenti di riga di comando) il descrittore della shared memory (`shm_id`) e l'indice della stazione a cui è stato assegnato dal coordinatore (`target_station`). Si collega alla shared memory, imposta l'handler per il segnale `SIGUSR1` (necessario per interrompere le letture bloccanti dalle code messaggi) e inizializza il proprio seme per la generazione di numeri casuali basandosi sul PID.
2. **Sincronizzazione Giornaliera:** All'inizio di ogni giornata, l'operatore:
  - a) Segnala di essere pronto incrementando il semaforo `SEM_READY` (`sem_op(+1)`).
  - b) Si blocca in attesa del segnale di avvio della giornata sul semaforo `SEM_DAY_START` (`sem_op(-1)`).
3. **Presa di Servizio alla Stazione:** Acquisisce un posto fisico presso la stazione target eseguendo una `sem_op(-1)` sul semaforo associato alla stazione (`SEM_SEATS_*`). Una volta ottenuto il posto, accede alla shared memory per incrementare in mutua esclusione (`SEM_Mutex_SHM`) il contatore degli operatori attivi alla stazione (`active_operators`).
4. **Ciclo di Servizio:** Fino a quando la giornata è attiva (`day_running == 1`), l'operatore esegue un ciclo continuo in cui:
  - a) **Attesa dell'Ordine:** Esegue una chiamata bloccante `msgrcv()` sulla coda messaggi della propria stazione.
  - b) **Erogazione del Servizio:** Se la stazione serve cibo (primi/secondi), controlla la disponibilità delle porzioni per il piatto richiesto sotto la protezione del mutex delle porzioni (`SEM_Mutex_PORZIONI_P` o `SEM_Mutex_PORZIONI_S`). Se disponibile, decrementa la porzione, simula il tempo di servizio eseguendo una sospensione proporzionale al tempo medio (`AVG_SRVC_*`) e risponde all'utente con successo (`served = 1`). Se le porzioni sono esaurite, risponde immediatamente all'utente segnalando il fallimento (`served = 0`) senza simulare alcun tempo di servizio.
  - c) **Erogazione Caffè e Cassa:** Nel caso del caffè (porzioni illimitate) o della cassa, l'operatore simula il servizio e risponde sempre con successo. Per la cassa, l'operatore calcola inoltre l'importo totale dovuto basandosi sui piatti consumati dal cliente ed aggiorna in mutua esclusione la statistica degli incassi giornalieri.
  - d) **Invio Risposta:** La risposta (`ServedMsg`) viene inviata sulla coda di risposta comune con tipo di messaggio (`mtype`) impostato al PID dell'utente richiedente.
  - e) **Gestione delle Pause:** Dopo ogni servizio completato, l'operatore calcola una probabilità del 15% di prendere una pausa. La pausa è consentita solo se ci sono ancora pause disponibili nella sua quota giornaliera (`nof_pause > 0`) e se la stazione non viene lasciata sguarnita (ovvero se vi è almeno un altro operatore

attivo nella medesima stazione). Durante la pausa, l'operatore decrementa il contatore degli operatori attivi in shared memory, rilascia temporaneamente il posto fisico della stazione (`sem_op(+1)` sul relativo semaforo), simula la pausa (`WORKER_PAUSE` minuti simulati) e infine riacquisisce il posto (`sem_op(-1)`) ed aggiorna lo stato.

5. **Chiusura della Giornata:** Quando il coordinatore imposta `day_running = 0` e invia il segnale `SIGUSR1`, la chiamata bloccante `msgrcv()` dell'operatore fallisce con errore `EINTR`. L'operatore rileva che la giornata è conclusa, esce dal ciclo di servizio, decrementa gli operatori attivi, rilascia il posto stazione ed attende la giornata successiva o la terminazione.

## 2.3 Ciclo di Vita dell'Utente

I processi `utente` simulano i clienti della mensa che entrano, scelgono il cibo, si mettono in coda alle stazioni, pagano ed infine consumano il pasto ai tavoli. Il loro ciclo di vita comprende le seguenti fasi:

1. **Inizializzazione:** Riceve all'avvio il descrittore della shared memory (`shm_id`). Si collega alla memoria condivisa, registra il proprio handler per `SIGUSR1` e ricava il proprio PID.
2. **Sincronizzazione Giornaliera:** Come gli operatori, partecipa alla barriera di inizio giornata incrementando `SEM_READY` e attendendo su `SEM_DAY_START`.
3. **Scelta del Menù:** All'avvio della giornata, l'utente genera le proprie preferenze casuali per il cibo e decide se consumare o meno il caffè.
4. **Routine di Servizio:** L'utente visita sequenzialmente le stazioni di cibo di suo interesse (primi, secondi, caffè) scegliendo l'ordine di visita in base alle lunghezze delle code al fine di minimizzare il tempo d'attesa.
5. **Pagamento e Consumazione:** Una volta terminato il giro delle stazioni alimentari, l'utente si sposta alla cassa per saldare il conto ed infine tenta di sedersi ad un tavolo per mangiare.
6. **Uscita:** Concluso il pasto (o in caso di fallimento del servizio per mancanza di cibo o chiusura anticipata), l'utente aggiorna le statistiche globali (tempo di attesa, stato del servizio) e ritorna alla barriera per la giornata successiva.

## 2.4 Algoritmi di Scelta e Comportamento dell'Utente

L'utente adotta strategie specifiche per l'ordinamento delle preferenze, il bilanciamento del carico tra le code e la gestione delle situazioni anomale.

### 2.4.1 Scelta del Menù e Preferenze (Algoritmo Fisher-Yates)

All'inizio della giornata, l'utente determina le proprie preferenze per ciascuna tipologia di piatto. Poiché il menù offre molteplici varianti per i primi e i secondi piatti (lette dal file `conf/menu.txt`), l'utente deve decidere l'ordine in cui preferisce consumarle. Questo ordinamento viene generato in modo casuale ed imparziale inizializzando un array con

gli indici consecutivi dei piatti disponibili (da 0 a  $N - 1$ ) e applicando l'algoritmo di **Fisher-Yates shuffle**:

```

1 void fisher_yates_shuffle(int *array, int n) {
2     for (int i = n - 1; i > 0; i--) {
3         int j = rand_r(&seed) % (i + 1);
4         int temp = array[i];
5         array[i] = array[j];
6         array[j] = temp;
7     }
8 }

```

Listing 1: Algoritmo di Fisher-Yates per la scelta dei piatti.

Questo garantisce che ogni possibile combinazione di preferenze abbia la stessa probabilità di essere generata, modellando comportamenti d'acquisto realistici. Successivamente, viene calcolata una probabilità del 25% per determinare se l'utente desidera consumare anche il caffè/dolce (`want_coffee = 1`).

#### 2.4.2 Accodamento Dinamico Bilanciato (Shortest Queue First)

Durante la permanenza nella mensa, l'utente deve visitare le stazioni pianificate: primi piatti, secondi piatti e, se desiderato, caffè. Per evitare colli di bottiglia e ottimizzare il flusso all'interno del locale, l'utente non segue un ordine fisso di stazioni (es. prima primi, poi secondi), ma valuta dinamicamente lo stato delle code in quel momento.

L'utente esamina le code delle stazioni non ancora visitate e sceglie di accodarsi presso quella che presenta la lunghezza della coda più corta in assoluto (*Shortest Queue First*). Per evitare race condition in cui più utenti scelgono contemporaneamente la stessa stazione alterandone lo stato prima che questo sia registrato, la lettura delle lunghezze delle code (`queue_length`) e l'incremento del contatore della stazione scelta avvengono in modo strettamente atomico all'interno di una sezione critica protetta dal semaforo mutex globale della shared memory `SEM_MUTEX_SHM`:

```

1 int min_queue = INT_MAX;
2 int target_station = -1;
3 sem_op(shm->sem_id, SEM_MUTEX_SHM, -1, SEM_UNDO);
4 for (int i = 0; i < 3; i++) {
5     if (station_done[i] == 0) {
6         int station_queue = shm->stations[i].queue_length;
7         if (station_queue < min_queue) {
8             min_queue = station_queue;
9             target_station = i;
10        }
11    }
12 }
13 if (target_station != -1) {
14     shm->stations[target_station].queue_length++;
15 }
16 sem_op(shm->sem_id, SEM_MUTEX_SHM, +1, SEM_UNDO);

```

Listing 2: Selezione della stazione con coda minore.

#### 2.4.3 Comportamento in caso di Esaurimento Porzioni

Quando l'utente giunge in testa alla coda di una stazione di cibo, invia un messaggio di richiesta (`ServingMsg`) indicando la sua prima preferenza (l'elemento a indice 0 dell'array

delle preferenze). L'operatore verifica se vi sono porzioni disponibili per tale piatto. Possono verificarsi due scenari:

- **Porzione disponibile:** L'operatore decrementa il contatore delle porzioni in shared memory, serve l'utente (simulando il tempo di servizio) e risponde con successo (`served = 1`).
- **Porzione esaurita:** L'operatore risponde immediatamente con un fallimento (`served = 0`) senza simulare attese. L'utente riceve il rifiuto e, rimanendo in coda, invia immediatamente una nuova richiesta per il piatto successivo nella propria lista di preferenze (l'elemento a indice 1).

Questo ciclo di tentativi prosegue finché l'utente non ottiene un piatto con successo o non esaurisce tutte le opzioni disponibili per quella stazione. Se tutte le opzioni del menù per quella stazione sono esaurite (ad esempio sono stati esauriti tutti i tipi di primi piatti), l'utente abbandona la stazione senza aver ottenuto cibo (`ordered[station] = 0`).

#### 2.4.4 Comportamento in caso di Mancato Servizio (Not Served Path)

La politica della mensa impone che un utente debba consumare un pasto principale per poter completare la sua routine. Se l'utente, dopo aver tentato il servizio alle stazioni dei primi e dei secondi piatti, si ritrova con **entrambi i piatti non serviti** (`ordered[STATION_PRIMI] == 0 && ordered[STATION_SECONDI] == 0`), decide di non recarsi alla cassa e di non occupare un tavolo. L'utente abbandona quindi la mensa e registra l'evento aggiornando in mutua esclusione la statistica dei clienti non serviti tramite la funzione `stats_record_user_not_served()`.

Analogamente, se l'utente ha ottenuto almeno un piatto ma la transazione di pagamento alla cassa fallisce (ad esempio perché la coda messaggi della cassa viene distrutta a fine giornata prima che l'operatore possa elaborare la transazione), l'utente non può consumare il cibo. Anche in questo caso si attiva il percorso di fallimento, registrando l'utente come non servito e liberando le risorse.

#### 2.4.5 Consumazione al Tavolo

Se l'utente ha ottenuto con successo almeno un piatto principale e ha completato con successo il pagamento alla cassa, procede verso l'area tavoli. L'utente tenta di acquisire un posto eseguendo una `sem_op(-1)` sul semaforo contatore `SEM_TABLE_SEATS`, inizializzato alla capacità massima dei tavoli. Se tutti i posti sono occupati, il processo utente si blocca finché un tavolo non si libera. Una volta seduto, l'utente simula il tempo necessario per consumare il pasto tramite una `sim_sleep()` proporzionale al numero di piatti effettivamente ottenuti (2 secondi simulati per piatto). Al termine, rilascia il posto con una `sem_op(+1)` e registra il successo del servizio aggiornando le statistiche complessive.

#### 2.4.6 Chiusura Anticipata della Giornata (Fine Servizio)

La giornata lavorativa della mensa ha una durata temporale definita. Quando la giornata giunge al termine, il coordinatore imposta il flag globale `day_running = 0` in shared memory e invia il segnale `SIGUSR1` a tutti i processi figli attivi. Gli utenti gestiscono questa interruzione in modo sicuro controllando il flag di uscita e lo stato della giornata prima di effettuare operazioni bloccanti (come l'invio di un messaggio sulla coda o l'attesa di una risposta):

```

1 if (should_exit || !shm->day_running) {
2     break;
3 }

```

Listing 3: Controllo di terminazione giornaliera nel ciclo utente.

Se il segnale viene ricevuto mentre l'utente è in attesa di un messaggio, la chiamata di sistema viene interrotta (**EINTR**), la condizione di uscita viene rilevata, e l'utente abbandona le code decrementando correttamente la lunghezza delle code in shared memory (**leave\_queue()**) grazie alla protezione **SEM\_UNDO**. In questo modo si evitano stalli o blocchi indefiniti dei processi e si garantisce una transizione pulita verso la giornata successiva o verso lo shutdown definitivo della simulazione.

## 3 Meccanismi IPC Scelti e Motivazioni

La comunicazione inter-processo si basa interamente su **System V IPC**, come richiesto dalle specifiche. Le tre primitive utilizzate sono:

### 3.1 Shared Memory

Un singolo segmento di shared memory contiene la struttura **SharedData**, che include:

- Flag di controllo (**simulation\_running**, **day\_running**, **current\_day**, **termination\_cause**).
- Stato delle stazioni (**StationData stations[4]**): lunghezza coda, operatori attivi, porzioni rimanenti, tempi di servizio.
- Identificatori delle code di messaggi e dei semafori.
- Statistiche della simulazione (**SimStats**).
- Configurazione (**Config**).

**Motivazione:** la shared memory è il meccanismo più efficiente per condividere grandi quantità di dati strutturati tra processi. Lo stato delle stazioni, le statistiche e la configurazione devono essere accessibili da tutti i processi con latenza minima.

### 3.2 Semafori

Viene creato un set di **11 semafori** con le seguenti funzioni:

Tabella 1: Set di semafori System V.

| Indice | Nome                 | Scopo                                                               |
|--------|----------------------|---------------------------------------------------------------------|
| 0      | SEM_MUTEX_SHM        | Mutua esclusione per accesso in scrittura a <code>SharedData</code> |
| 1      | SEM_SEATS_PRIMI      | Contatore posti operatore alla stazione primi                       |
| 2      | SEM_SEATS_SECONDI    | Contatore posti operatore alla stazione secondi                     |
| 3      | SEM_SEATS_COFFEE     | Contatore posti operatore alla stazione caffè                       |
| 4      | SEM_SEATS_CASSA      | Contatore posti operatore alla cassa                                |
| 5      | SEM_TABLE_SEATS      | Contatore posti tavolo disponibili                                  |
| 6      | SEM_READY            | Barriera di sincronizzazione figli                                  |
| 7      | SEM_DAY_START        | Segnale di inizio giornata                                          |
| 8      | SEM_MUTEX_STATS      | Mutua esclusione per aggiornamento statistiche                      |
| 9      | SEM_MUTEX_PORZIONI_P | Mutex per decremento porzioni primi                                 |
| 10     | SEM_MUTEX_PORZIONI_S | Mutex per decremento porzioni secondi                               |

**Motivazione:** i semafori System V permettono di implementare sia mutex binari (inizializzati a 1) sia contatori di risorse (inizializzati alla capacità). Il flag `SEM_UNDO` garantisce il rilascio automatico in caso di terminazione anomala, prevenendo deadlock residui.

### 3.3 Code di Messaggi

Sono utilizzate **5 code di messaggi**:

- **4 code di richiesta** (una per stazione): gli utenti inviano ordini (`ServingMsg` o `OrderMsg`) alla coda della stazione desiderata.
- **1 coda di risposta** (comune): gli operatori inviano le risposte (`ServedMsg`) indirizzate al PID dell'utente tramite il campo `mtype`.

**Motivazione:** le code di messaggi garantiscono un disaccoppiamento naturale tra produttore (utente) e consumatore (operatore). Il filtraggio per `mtype` permette di instradare le risposte al corretto processo utente senza strutture dati condivise aggiuntive.

Le code vengono **ricreate ogni giornata** per evitare che messaggi residui della giornata precedente inquinino la successiva.

## 4 Politica di Assegnazione Operatori

Gli operatori vengono assegnati alle stazioni all'avvio della simulazione secondo una politica a due fasi implementata nella funzione `get_target_station()`:

## 4.1 Fase 1: Assegnazione Garantita

I primi `NUM_STATIONS` (4) operatori vengono assegnati uno per stazione, in ordine di priorità decrescente basato sul tempo medio di servizio (`avg_srvc`). La stazione più lenta riceve il primo operatore.

## 4.2 Fase 2: Distribuzione Bilanciata

Gli operatori successivi vengono distribuiti con un criterio di bilanciamento pesato. Per ciascun operatore aggiuntivo, si sceglie la stazione che massimizza il rapporto:

$$\text{best} = \arg \max_j \frac{\text{avg\_srvc}[j]}{\text{assigned}[j]}$$

Questo approccio garantisce che le stazioni con tempi di servizio più elevati ricevano proporzionalmente più operatori.

## 4.3 Ordinamento per Priorità

L'ordinamento viene effettuato tramite la funzione `sort_station_priority()`, che implementa un *selection sort* sull'array di indici delle stazioni, producendo un vettore di indirizzamento indiretto che non modifica l'array originale in shared memory.

# 5 Gestione della Sincronizzazione e della Concorrenza

La sincronizzazione è il cuore del sistema e si articola su più livelli:

## 5.1 Barriera di Sincronizzazione

Il coordinatore e i processi figli si sincronizzano tramite una barriera a due fasi implementata con i semafori `SEM_READY` e `SEM_DAY_START`:

1. Ogni figlio incrementa `SEM_READY` (+1) per segnalare la propria disponibilità.
2. Il coordinatore esegue `sem_op(SEM_READY, -TOTAL_CHILDREN)` per attendere che tutti i figli siano pronti.
3. Il coordinatore prepara la giornata e poi esegue `sem_op(SEM_DAY_START, +TOTAL_CHILDREN)` per sbloccare tutti i figli simultaneamente.
4. Ogni figlio esegue `sem_op(SEM_DAY_START, -1)` per attendere il via.

Questo meccanismo garantisce che nessun figlio inizi ad operare prima che tutte le risorse del giorno siano pronte.

## 5.2 Sezioni Critiche

L'accesso concorrente alla shared memory è protetto da mutex dedicati:

- **SEM\_MUTEX\_SHM**: protegge lo stato globale (flag di controllo, lunghezze code, contatori operatori).
- **SEM\_MUTEX\_STATS**: protegge gli aggiornamenti delle statistiche (**SimStats**).
- **SEM\_MUTEX\_PORZIONI\_P/S**: proteggono il decremento delle porzioni per primi e secondi, separatamente dal mutex principale per ridurre la contesa.

## 5.3 Gestione delle Risorse con Semafori Contatore

I semafori **SEM\_SEATS\_\*** e **SEM\_TABLE\_SEATS** agiscono come contatori di risorse limitate:

- Gli operatori eseguono **sem\_op(-1)** per acquisire un posto alla stazione e **sem\_op(+1)** per rilasciarlo.
- Gli utenti eseguono **sem\_op(-1)** su **SEM\_TABLE\_SEATS** per sedersi al tavolo.
- I valori iniziali corrispondono alle capacità configurate.

## 5.4 Bilanciamento del Carico Utente

Gli utenti selezionano la stazione con la coda più corta (*shortest queue first*) tramite lettura atomica dei contatori **queue\_length** protetta da **SEM\_MUTEX\_SHM**. L'incremento della coda avviene nella stessa sezione critica per evitare race condition.

## 5.5 Pause Operatore

Un operatore può prendere una pausa solo se:

- Ci sono almeno 2 operatori attivi alla stazione (per non lasciare la stazione scoperta).
- Ha ancora pause disponibili (**nof\_pause > 0**).
- La giornata è ancora in corso (**day\_running**).

Durante la pausa, l'operatore rilascia il proprio posto alla stazione (**sem\_op(+1)**) e lo riacquisisce al termine (**sem\_op(-1)**).

# 6 Gestione della Terminazione

La simulazione può terminare per tre cause distinte:

## 6.1 Timeout (Causa 0)

La simulazione ha completato tutti i **SIM\_DURATION** giorni configurati. Il coordinatore esce dal ciclo giornaliero e avvia lo shutdown graceful.

## 6.2 Overload (Causa 1)

A fine giornata, il coordinatore somma le lunghezze di tutte le code e confronta il totale con `OVERLOAD_THRESHOLD`. Se il numero di utenti in attesa supera la soglia:

```
1 int total_queued = 0;
2 for (int j = 0; j < NUM_STATIONS; j++) {
3     total_queued += shm->stations[j].queue_length;
4 }
5 if (total_queued > shm->config.overload_threshold) {
6     shm->termination_cause = 1;
7     shm->simulation_running = 0;
8 }
```

## 6.3 Segnale Esterno — SIGINT (Causa 2)

Il coordinatore installa un handler per `SIGINT` e `SIGTERM` che:

1. Imposta `simulation_running = 0`.
2. Invia `SIGINT` a tutti i processi figli.
3. Attende la terminazione dei figli con `waitpid()`.
4. Invoca `exit()`, che attiva `atexit(cleanup_ipc)` per la pulizia delle risorse IPC.

## 6.4 Shutdown Graceful

La procedura di shutdown (`execute_graceful_shutdown()`) garantisce una terminazione ordinata:

1. Il coordinatore imposta `simulation_running = 0`.
2. Invia `SIGUSR1` a tutti i figli per interrompere eventuali `msgrcv()` bloccanti.
3. Incrementa `SEM_DAY_START` per sbloccare i figli in attesa della prossima giornata.
4. I figli controllano `simulation_running` e terminano.
5. Il coordinatore raccoglie tutti i figli con `waitpid()`.
6. Le risorse IPC vengono rilasciate da `cleanup_ipc()` registrata con `atexit()`.

# 7 Struttura dei File di Configurazione

## 7.1 Formato Generale

I parametri di configurazione vengono letti da file con formato `CHIAVE = VALORE`, uno per riga. Le righe vuote e i commenti (prefisso `#`) vengono ignorati. Il parser è implementato in `config.c` tramite la funzione `parse_config()`.

Se non viene specificato un file da riga di comando, il sistema cerca la variabile d'ambiente `MENSA_CONFIG`; in sua assenza, utilizza il percorso predefinito `conf/config.conf`.

## 7.2 Parametri di Configurazione

Tabella 2: Parametri di configurazione disponibili.

| Parametro          | Descrizione                                          |
|--------------------|------------------------------------------------------|
| SIM_DURATION       | Durata della simulazione in giorni                   |
| N_NANO_SECS        | Nanosecondi reali per minuto simulato                |
| NOF_WORKERS        | Numero totale di operatori                           |
| NOF_USERS          | Numero totale di utenti                              |
| NOF_WK_SEATS_*     | Capacità coda per ciascuna stazione                  |
| NOF_TABLE_SEATS    | Posti tavolo disponibili                             |
| AVG_SRVC_*         | Tempo medio di servizio per stazione (sec. simulati) |
| AVG_REFILL_*       | Porzioni iniziali/rifornimento per primi e secondi   |
| MAX_PORZIONI_*     | Limite massimo porzioni per tipo                     |
| NOF_PAUSE          | Pause consentite per operatore                       |
| PRICE_*            | Prezzi per piatto                                    |
| OVERLOAD_THRESHOLD | Soglia utenti in coda per terminazione overload      |
| MENU_FILE          | Percorso del file menù                               |

## 7.3 File Menù

Il file `conf/menu.txt` definisce i piatti disponibili con il formato `TIPO:NOME_PIATTO`. I tipi riconosciuti sono `PRIMO`, `SECONDO`, `COFFEE` e `DOLCE`. Il parser conta il numero di varianti per primi e secondi, determinando la dimensione degli array `portion_left[]`.

## 7.4 Configurazioni di Consegna

Sono fornite due configurazioni specifiche, come richiesto dalle specifiche:

`config_timeout.conf` — Configurazione che produce terminazione per **timeout**. Imposta `OVERLOAD_THRESHOLD = 999` (irraggiungibile), servizi veloci e un numero moderato di utenti (10), garantendo che la simulazione completi tutti e 3 i giorni previsti senza mai raggiungere la soglia di overload.

`config_overload.conf` — Configurazione che produce terminazione per **overload**. Imposta `OVERLOAD_THRESHOLD = 5` e tempi di servizio molto elevati (500 sec simulati per primi e secondi) con 100 utenti. Il numero di utenti in attesa alla fine del primo giorno supera ampiamente la soglia, causando terminazione immediata.

# 8 Compilazione e Utilizzo

## 8.1 Requisiti

- GCC con supporto C99/C11
- GNU Make
- Sistema operativo Linux con supporto System V IPC

## 8.2 Compilazione

```
1 make clean all
```

Listing 4: Compilazione del progetto.

Il Makefile compila tre eseguibili in bin/: `responsabile_mensa`, `operatore`, `utente`. I flag di compilazione includono `-Wvla -Wextra -Werror` per garantire zero warning.

## 8.3 Esecuzione

```
1 # Esecuzione con config di default
2 make run
3
4 # Terminazione per timeout
5 make run CONF=config/config_timeout.conf
6
7 # Terminazione per overload
8 make run CONF=config/config_overload.conf
9
10 # Esecuzione diretta
11 ./bin/responsabile_mensa config/config_timeout.conf
```

Listing 5: Esecuzione con configurazione specifica.

## 8.4 Pulizia Risorse IPC

In caso di terminazione anomala (es. `kill -9`), le risorse IPC potrebbero rimanere allocate. Per verificare e rimuoverle:

```
1 # Verifica risorse IPC
2 ipcs
3
4 # Rimozione manuale
5 ipcrm -a
```

Listing 6: Pulizia risorse IPC residue.

# 9 Struttura del Progetto

```
1 S0_Project/
2 |-- Makefile
3 |-- conf/
4 |   |-- config.conf           # Configurazione di default
5 |   |-- config_timeout.conf   # Terminazione per timeout
6 |   |-- config_overload.conf  # Terminazione per overload
7 |   '-- menu.txt              # Menu della mensa
8 |-- include/
9 |   |-- common.h              # Costanti e macro globali
10 |   |-- config.h              # Struttura Config e parser
11 |   |-- ipc_utils.h           # Wrapper IPC System V
12 |   |-- shared_data.h         # SharedData e StationData
13 |   |-- stats.h               # SimStats e funzioni report
14 |   '-- time_utils.h          # Conversioni tempo simulato
```

```

15 | -- src/
16 |   |-- responsabile_mensa.c      # Processo coordinatore
17 |   |-- operatore.c              # Processo operatore
18 |   |-- utente.c                 # Processo utente
19 |   |-- config.c                 # Parser configurazione
20 |   |-- ipc_utils.c              # Implementazione wrapper IPC
21 |   |-- time_utils.c             # Implementazione tempo
22 |   '-- stats.c                  # Implementazione statistiche
23 |-- bin/                         # Eseguibili compilati
24 |-- build/                       # File oggetto compilati
25 '-- relazione_finale/
26   '-- relazione.tex              # Questa relazione

```

Listing 7: Struttura della directory del progetto.

## 10 Conclusioni

Il progetto implementa con successo una simulazione multi-processo di una mensa universitaria utilizzando esclusivamente meccanismi IPC System V. Le scelte architetturali principali — un singolo segmento di shared memory, semafori con `SEM_UNDO` per la robustezza, e code di messaggi con filtraggio per PID — garantiscono correttezza, assenza di deadlock e pulizia automatica delle risorse anche in caso di terminazione anomala.

La separazione dei mutex (generale, statistiche, porzioni) riduce la contesa tra processi, mentre la politica di assegnazione pesata degli operatori ottimizza i tempi di servizio. Le due configurazioni fornite dimostrano il corretto funzionamento di entrambe le condizioni di terminazione previste dalle specifiche.